Program Studi Teknik Informatika
Fakultas Teknologi Industri, Institut Teknologi Sumatera

IF25-11002 PRAKTIKUM PEMROGRAMAN

Modul Belajar 5

Perulangan Bagian 1

for, while, dan do-while dengan pola pencacah, akumulator, dan sentinel

Tim Pengajar
Pendamping Pertemuan 5. Tahun 2026

Komponen	Keterangan
Pertemuan	5 dari 16
Capaian	CPMK0607 (menerapkan) dan CPMK0608 (menjelaskan) pada konsep dasar algoritma dan sistem komputer
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P5 - Perulangan 1 - Hands-on/`

MODUL 5

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: Rata-rata nilai satu kelas
3	Langkah 1: Perulangan for
4	Langkah 2: Pencacah dan akumulator
5	Langkah 3: Perulangan while
6	Langkah 4: Nilai sentinel
7	Langkah 5: Perulangan do-while
8	Langkah 6: Memilih jenis perulangan
9	Pesan error yang sering muncul
10	Latihan mandiri
11	Rangkuman
12	Cek pemahaman
13	Exit Ticket dan tugas
14	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini mendampingi Pertemuan 5. Program kita sudah bisa memilih. Minggu ini program belajar mengulang: menjalankan blok yang sama berkali-kali tanpa menulis ulang kodenya, sambil mengumpulkan hasil di setiap putaran.

Isinya sama dengan slide di kelas, tetapi ditulis lebih pelan dan lebih lengkap, supaya kalian bisa mengulang sendiri di rumah tanpa harus menebak maksud slide.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menulis program yang mengulang proses dengan `for`, `while`, dan `do-while`, memakai pola pencacah, akumulator, dan nilai sentinel (Sub-CPMK 5.1, CPMK0607)	Langkah 1 sampai 6
2	Menelusuri perulangan dengan trace table, menentukan berapa kali badan perulangan dijalankan, dan menjelaskan penyebab off-by-one atau perulangan tak berhenti (Sub-CPMK 5.2, CPMK0608)	kotak Tebak dulu, Latihan 3

Karena setiap instrumen penilaian dibagi rata antara Bagian A (menulis kode) dan Bagian B (menelusuri dan menjelaskan), yang dinilai bukan hanya program kalian jalan, tetapi juga kalian bisa menjelaskan mengapa program itu jalan.

Cara memakai modul ini

Setiap langkah punya pola yang sama: penjelasan konsep, kadang contoh dari kehidupan sehari-hari, lalu kode yang bisa langsung kalian ketik. Sesudah kode ada kotak **Tebak dulu**. Tulis tebakan kalian di kertas sebelum membaca **Jawaban** di bawahnya. Kebiasaan menebak lalu membuktikan inilah yang membuat konsep menempel, bukan sekadar membaca.

> PENTING **Ketik, jangan salin**
>
> Ketik ulang setiap contoh di GDB Online atau editor kalian, jangan disalin-tempel. Saat mengetik, kalian akan membuat salah ketik, membaca pesan error, dan memperbaikinya. Itu bagian dari belajar.

Yang perlu disiapkan

Peramban dengan akses ke onlinegdb.com, atau g++ di komputer kalian sendiri. Kertas dan alat tulis untuk menebak keluaran dan membuat trace table. Folder hands-on pertemuan ini dari repo mata kuliah.

PEMBUKA

Masalah pembuka: Rata-rata nilai satu kelas

Asisten praktikum harus menghitung rata-rata nilai Exit Ticket. Jumlah mahasiswa berbeda di setiap kelas, kadang 30, kadang 45.

Menulis `cin` sebanyak 45 kali jelas tidak masuk akal. Lagi pula banyaknya data baru diketahui saat program berjalan.

Tampilan yang diinginkan

```
Masukkan nilai (-1 untuk selesai):
70 85 60 90 -1
Banyak    : 4
Tertinggi : 90
Rata-rata : 76.25
```

TEBAK DULU **Coba pikirkan**

Langkah apa yang terus diulang untuk setiap nilai? Bagaimana program tahu kapan berhenti?

JAWABAN **Cara memecahnya**

Temukan yang berulang: Baca satu nilai, tambahkan ke total, tambah pencacah. Tentukan kapan berhenti: Berhenti saat bertemu -1, atau setelah n kali. Terjemahkan: `while` untuk syarat berhenti, `for` untuk banyak putaran yang sudah diketahui.

LANGKAH 1

Perulangan for

`for (awal; syarat; perubahan)` cocok bila banyak putaran sudah diketahui. Urutannya: jalankan bagian awal sekali, periksa syarat, jalankan badan, jalankan perubahan, periksa syarat lagi, dan seterusnya sampai syaratnya salah.

Variabel `i` yang dideklarasikan di dalam `for` hanya hidup di dalam perulangan itu.

ANALOGI **Contoh sehari-hari**

Seperti absensi kelas: dosen memanggil nomor 1, lalu 2, dan seterusnya sampai nomor terakhir. Banyak putarannya sudah diketahui sejak awal.

Bentuk umum

```
for (awal; syarat; perubahan) {
    // diulang selama syarat benar
}
for (int i = 1; i <= n; i++) { ... }
```

for.cpp

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
cout << endl;
for (int i = 10; i > 0; i -= 3) {
    cout << i << " ";
}
cout << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
1 2 3 4 5
10 7 4 1
```

Tiga bagian `for`: nilai awal; syarat lanjut; perubahan. Syarat diperiksa sebelum setiap putaran.

LANGKAH 2

Pencacah dan akumulator

Dua pola paling sering dipakai. Akumulator mengumpulkan nilai (`jumlah += i`), pencacah menghitung kejadian (`genap++`). Keduanya harus diberi nilai awal sebelum perulangan. Bila diberi nilai awal di dalam perulangan, nilainya ter-reset di setiap putaran.

ANALOGI **Contoh sehari-hari**

Akumulator seperti celengan yang diisi setiap hari; pencacah seperti turus di papan untuk menghitung kejadian. Keduanya harus kosong sebelum mulai.

Bentuk umum

```
int total = 0, cacah = 0;   // sebelum perulangan
// di dalam perulangan:
total += nilai;
cacah++;
```

akumulator.cpp

```
int jumlah = 0, genap = 0;
for (int i = 1; i <= 10; i++) {
    jumlah += i;
    if (i % 2 == 0) {
        genap++;
    }
}
cout << "Jumlah: " << jumlah << endl;
cout << "Genap : " << genap << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
Jumlah: 55
Genap : 5
```

Akumulator dan pencacah selalu diberi nilai awal sebelum perulangan, bukan di dalamnya.

LANGKAH 3

Perulangan while

`while (syarat)` cocok bila banyak putaran belum diketahui, hanya syarat berhentinya. Syarat diperiksa sebelum setiap putaran, sehingga badan bisa saja tidak dijalankan sama sekali.

Pastikan ada pernyataan di dalam badan yang membuat syaratnya suatu saat salah. Bila lupa, perulangan tidak berhenti. Di GDB Online, tekan Stop untuk menghentikannya.

ANALOGI **Contoh sehari-hari**

Seperti menabung sampai uang cukup untuk membeli laptop: kita tidak tahu berapa bulan, yang kita tahu hanya kapan berhenti.

Bentuk umum

```
while (syarat) {
    // diulang selama syarat benar
    // ubah sesuatu agar syarat suatu saat salah
}
```

while.cpp

```
int saldo = 100000, bulan = 0;
while (saldo < 500000) {
    saldo += 150000;
```

```
    bulan++;
}
cout << bulan << " bulan, saldo " << saldo << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
3 bulan, saldo 550000
```

Sesuatu di dalam badan harus mengubah syarat. Bila tidak, perulangan tidak pernah berhenti.

LANGKAH 4

Nilai sentinel

Sentinel adalah nilai khusus yang menandai akhir data, misalnya -1 untuk nilai ujian. Polanya: baca satu nilai sebelum perulangan, periksa di syarat `while`, proses, lalu baca nilai berikutnya di akhir badan. Urutan ini menjamin sentinel tidak ikut diproses.

ANALOGI **Contoh sehari-hari**

Seperti kasir yang terus memindai barang sampai pembeli meletakkan pembatas di ban berjalan. Pembatas itu sentinel: tanda berhenti, bukan barang yang dihitung.

Bentuk umum

```
cin >> x;
while (x != SENTINEL) {
    // proses x
    cin >> x;
}
```

sentinel.cpp

```cpp
int nilai, banyak = 0, total = 0;
cin >> nilai;
while (nilai != -1) {
    banyak++;
    total += nilai;
    cin >> nilai;
}
cout << banyak << " data, total " << total << endl;
```

Masukan

```
70 85 60 90 -1
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
4 data, total 305
```

Baca sekali sebelum perulangan, lalu baca lagi di akhir badan. Dengan begitu -1 tidak ikut dihitung.

LANGKAH 5

Perulangan do-while

`do { ... } while (syarat);` menjalankan badan dulu, baru memeriksa syarat. Badannya pasti jalan minimal sekali, sehingga cocok untuk menu dan validasi masukan.

ANALOGI **Contoh sehari-hari**

Seperti mesin ATM yang selalu meminta PIN minimal sekali, lalu baru memeriksa apakah PIN-nya benar.

Bentuk umum

```
do {
    // jalan minimal sekali
} while (syarat);
```

dowhile.cpp

```
int pin, coba = 0;
do {
    cin >> pin;
    coba++;
} while (pin != 1234 && coba < 3);
cout << (pin == 1234 ? "Masuk" : "Terkunci");
cout << " setelah " << coba << " kali" << endl;
```

Masukan

```
1111 1234
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
Masuk setelah 2 kali
```

Syarat diperiksa sesudah badan. Jangan lupa titik koma setelah `while (...)`.

LANGKAH 6

Memilih jenis perulangan

Secara teknis ketiga perulangan bisa saling menggantikan. Pilihan yang tepat membuat maksud program langsung terbaca.

ANALOGI **Contoh sehari-hari**

Seperti memilih kendaraan: ketiganya bisa membawa kita sampai, tetapi ada yang paling cocok untuk perjalanan tertentu.

Situasi	Pilihan	Contoh
Banyak putaran diketahui	`for`	tampilkan 1 sampai n
Berhenti karena syarat, bisa nol putaran	`while`	baca sampai sentinel
Badan harus jalan minimal sekali	`do-while`	menu, validasi PIN

PENTING

Ketiganya bisa saling menggantikan. Pilih yang paling jelas menyatakan maksud kalian.

WASPADA

Pesan error yang sering muncul

Semua pesan di tabel ini dihasilkan g++ dengan opsi `-Wall`, sama seperti yang dipakai `cek.sh`.

Kesalahan	Contoh	Pesan pertama dari g++
Koma di header for	`for (int i = 0, i < 5, i++)`	`error: expected ';' before '<' token`
Variabel di luar jangkauan	`cout << i; sesudah for`	`error: 'i' was not declared in this scope`
Lupa ; sesudah do-while	`} while (x < 5)`	`error: expected ';' before 'cout'`
Perbandingan bertipe beda	`i < s.length()` dengan `int`	`warning: comparison of integer expressions of different signedness: 'int' and ...`
Badan for kosong	`for (...);`	`warning: this 'for' clause does not guard...`

Off-by-one (kelebihan atau kekurangan satu putaran) dan perulangan tak berhenti tidak dilaporkan compiler. Keduanya hanya ketahuan lewat trace table atau pengujian, karena itu latihan tantangan minggu ini adalah menelusuri perulangan.

LATIHAN

Latihan mandiri

Latihan ini sama dengan hands-on di kelas. Semua file ada di `02 Hands-on/P5 - Perulangan 1 - Hands-on/latihan/`. Kerjakan berurutan, lalu cocokkan dengan `expected/` atau jalankan `bash cek.sh`.

No	File	Tingkat	Yang dilatih
1	`handson1-deret.cpp`	Dasar	`for`, akumulator
2	`handson2-tabungan.cpp`	Menengah	`while` dengan syarat berhenti
3	`handson3-rerata.cpp`	Tantangan	trace table, perbaiki tiga kesalahan sentinel
+	`bonus-faktorial.cpp`	Pengayaan	`do-while` untuk menu, `long long`

Latihan 1: handson1-deret.cpp

Tampilkan 1 sampai n dan jumlahnya. Masukan uji: `6`.

Latihan 2: handson2-tabungan.cpp

Simulasikan saldo tabungan per bulan sampai mencapai target.

Latihan 3: handson3-rerata.cpp

Program rata-rata dengan sentinel -1 memberi hasil salah. Buat trace table, perbaiki, dan jelaskan.

Bonus: bonus-faktorial.cpp

Menu berulang untuk faktorial dan jumlah deret sampai pengguna memilih 0.

Latihan menelusuri

tebak.cpp

```
int x = 0;
for (int i = 0; i < 4; i++) {
    x += i * 2;
}
int k = 1;
while (k < 20) {
    k *= 3;
}
```

```
cout << x << " " << k << endl;
```

TEBAK DULU

Tulis keluarannya di kertas tanpa menjalankan program. Buat trace table bila perlu.

JAWABAN

Keluaran

```
12 27
```

x = 0 + 2 + 4 + 6 = 12. k menjadi 3, 9, 27; berhenti saat 27 tidak lagi kurang dari 20.

PENUTUP

Rangkuman

Konsep	Yang perlu diingat
Perulangan for	Tiga bagian `for`: nilai awal; syarat lanjut; perubahan.
Pencacah dan akumulator	Akumulator dan pencacah selalu diberi nilai awal sebelum perulangan, bukan di dalamnya.
Perulangan while	Sesuatu di dalam badan harus mengubah syarat.
Nilai sentinel	Baca sekali sebelum perulangan, lalu baca lagi di akhir badan.
Perulangan do-while	Syarat diperiksa sesudah badan.
Memilih jenis perulangan	Ketiganya bisa saling menggantikan.

PENUTUP

Cek pemahaman

Jawab tanpa menjalankan program, lalu periksa dengan menjalankannya.

1. Berapa kali badan `for (int i = 2; i < 10; i += 2)` dijalankan?
2. Mengapa `int total = 0;` harus di luar perulangan?
3. Apa beda `while` dan `do-while` bila syaratnya sejak awal salah?
4. Apa keluaran `int k = 1; while (k < 50) k *= 2; cout << k;`?
5. Pada pola sentinel, mengapa `cin` ditulis di akhir badan?

JAWABAN

1. 4 kali (i = 2, 4, 6, 8).
2. Bila di dalam, total di-reset di setiap putaran.
3. `while` tidak menjalankan badan; `do-while` menjalankannya sekali.
4. 64.
5. Agar sentinel diperiksa di syarat sebelum sempat diproses.

PENILAIAN

Exit Ticket 5

Dikerjakan di 25 menit terakhir kelas, sendiri, tanpa AI, internet, atau diskusi. Exit Ticket 5 adalah satu dari 14 Exit Ticket; rata-ratanya menjadi komponen berbobot 25%.

Soal	Isi	Poin	CPMK
A1	Tulis program yang menampilkan kelipatan k dari 1 sampai n beserta banyaknya	25	CPMK0607
A2	Tulis program yang membaca bilangan sampai 0 lalu menampilkan jumlah bilangan positif	25	CPMK0607
B1	Buat trace table sebuah perulangan dan tentukan keluarannya	20	CPMK0608
B2	Jelaskan mengapa sebuah perulangan tidak berhenti dan cara memperbaikinya	20	CPMK0608
B3	Tentukan berapa kali badan perulangan dijalankan dan jelaskan alasannya	10	CPMK0608

PENILAIAN

Tugas 5: Program Statistik Nilai

Kumpulkan sebelum Pertemuan 6 lewat kanal pengumpulan yang diumumkan dosen.

Bagian A: program (50 poin)

Buat program statistik nilai dengan ketentuan berikut.

1. Baca jumlah siswa n, ulangi sampai n > 0
2. Baca nilai n siswa; setiap nilai harus 0 sampai 100, ulangi bila tidak valid
3. Tampilkan nilai tertinggi, nilai terendah, dan rata-rata kelas
4. Tampilkan jumlah siswa LULUS (nilai 75 ke atas) dan TIDAK LULUS
5. Menu `do-while` untuk mengulang atau keluar

> ANALOGI **Bonus, tidak wajib**
>
> Tampilkan persentase kelulusan dengan dua desimal.

Bagian B: penjelasan (50 poin)

Komentar maksimal 150 kata dan trace table untuk n = 3 dengan nilai 80, 70, 90. Jelaskan alasan memilih `for`, `while`, atau `do-while` di setiap bagian, dan satu kesalahan off-by-one atau perulangan tak berhenti yang kalian temui.

Rubrik Tugas 5

Kriteria	Poin	Sangat baik 85–100	Baik 70–84	Cukup 55–69	Kurang <55
A1 Program berjalan	20	tanpa error dan peringatan	ada peringatan	perlu satu perbaikan	tidak terkompilasi
A2 Validasi dan perulangan	15	validasi n dan nilai, menu berulang	satu validasi kurang	dua kurang	tanpa validasi
A3 Ketepatan statistik	15	semua tepat termasuk n = 1	satu salah	dua salah	sebagian besar salah
B1 Trace table dan alasan	30	trace lengkap dan alasan tepat	satu baris trace salah	trace tidak lengkap	tidak ada atau disalin
B2 Cerita error dan pengujian	20	pesan, sebab, perbaikan, uji	pesan tidak dikutip	hanya perbaikan	tidak ada

Nilai kriteria = poin × skor level. Contoh: A1 di level Baik dengan skor 80 memberi 20 × 0,80 = 16 poin.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

1. Deitel, P. dan Deitel, H. C++ How to Program, edisi ke-10, Bab 4 dan 5.
2. learncpp.com, Bab 8 (loops).
3. Gaddis, T. Starting Out with C++, Bab 5.
4. Slide `01 Slide/P5 Perulangan Bagian 1.pdf` dan hands-on di `02 Hands-on/P5 - Perulangan 1 - Hands-on/`.

> PENTING **Minggu depan**
>
> Pertemuan 6 membahas perulangan bersarang, `break`, `continue`, dan pola bintang, sejalan dengan Algoritma Pertemuan 6.